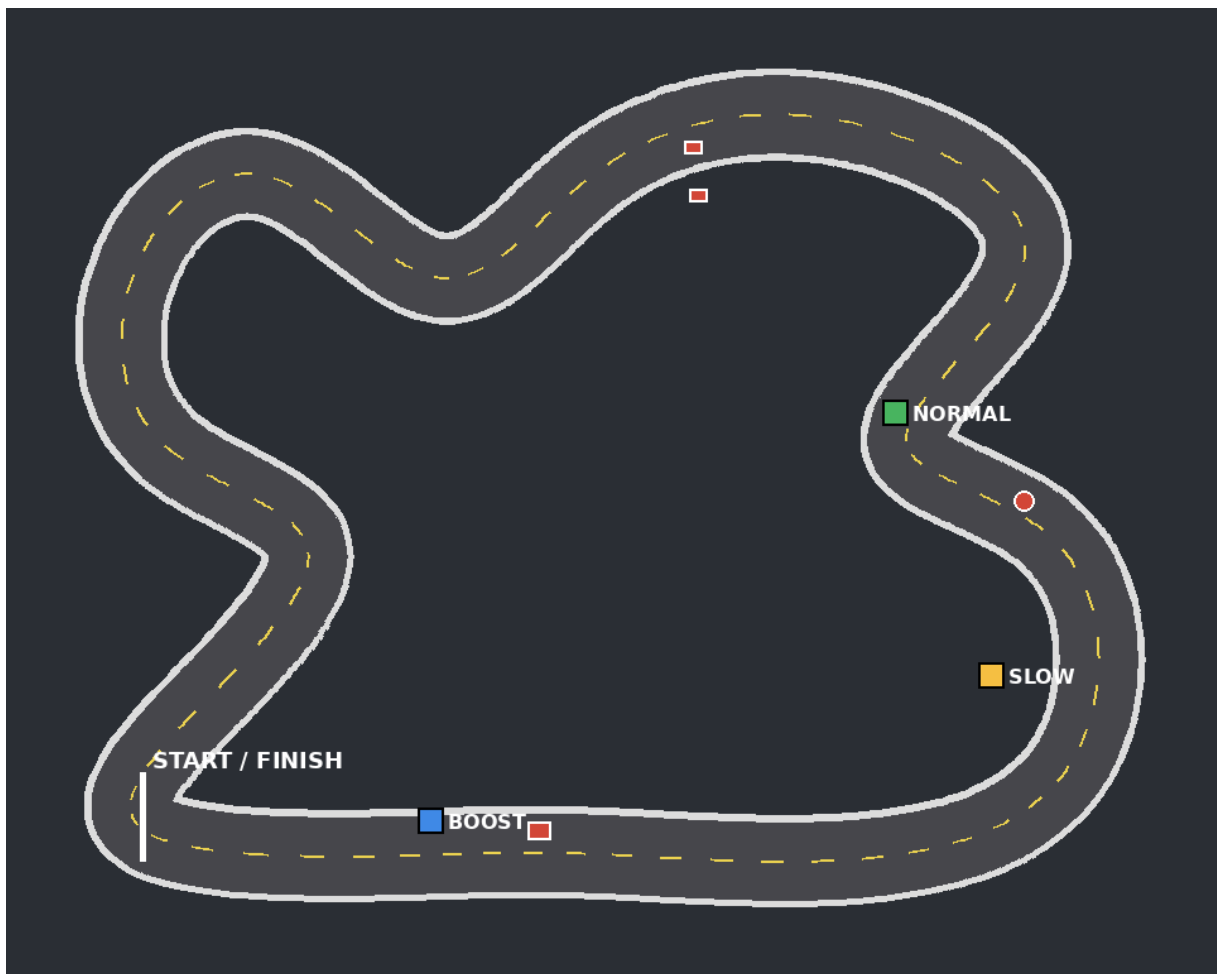


MuSHR Vision-Aware Autonomous Racing Challenge

A Five-Day Simulation-Only Assignment for Intermediate Robotics Students



Duration: 5 working days
Team size: 2–4 students
Platform: MuSHR-compatible simulation
Primary sensors: 2D LiDAR, odometry/localization, forward RGB camera
Control output: Ackermann steering and speed commands

Instructor-provided track, obstacle layouts, centre-line waypoints and visual sign assets are included with this handout.

Contents

1	Challenge Overview	2
2	Learning Outcomes	2
3	Provided Simulation Assets	2
3.1	Track characteristics	2
4	Computer-Vision Component	2
5	Recommended System Architecture	4
6	Obstacle Challenge	4
7	Five-Day Schedule and Checkpoints	4
8	Minimum Software Interface	5
9	Rules and Constraints	5
10	Evaluation Protocol	5
11	Grading Rubric	6
12	Required Submission	6
13	Suggested Report Questions	6
14	Instructor Notes	7
15	References	7

1 Challenge Overview

Design an autonomous racing stack that completes three consecutive laps on a known training track and generalizes to an evaluation configuration. The vehicle must race quickly, remain inside the track, avoid static obstacles, interpret visual speed signs and recover from common failures without manual control.

The task is intentionally simulation-only. Mapping and SLAM are outside the scope. Students receive an occupancy-grid map, an initial pose and centre-line waypoints. Their main work is control, local safety, computer vision, integration and quantitative evaluation.

Required capabilities

1. Follow the supplied track without teleoperation.
2. Increase speed on straights and brake before high-curvature sections.
3. Use LiDAR to avoid obstacles and prevent unsafe commands.
4. Detect track-side visual signs from a simulated front camera.
5. Apply the detected sign as a temporary speed-zone constraint.
6. Detect being stuck or off-course and execute an automatic recovery.
7. Complete three consecutive laps and produce reproducible logs.

2 Learning Outcomes

By the end of the assignment, students should be able to:

- implement and tune a geometric path-following controller;
- derive a speed profile from path curvature;
- process a **LaserScan** for emergency braking and local avoidance;
- build an OpenCV-based fiducial-marker detector;
- combine nominal control and safety constraints;
- implement a finite-state recovery controller;
- evaluate a robot using lap time, collision rate, sign accuracy and robustness metrics.

3 Provided Simulation Assets

The assignment package contains:

- `mushr_training_track.pgm` and `.yaml`: ROS-style occupancy grid;
- `centerline_waypoints.csv`: waypoint ID, x , y and heading;
- `obstacle_sets.yaml`: one public development layout and hidden evaluation layouts;
- `vision_sign_layouts.yaml`: public and hidden sign placements;
- three ArUco sign-board textures: SLOW, NORMAL and BOOST;
- an annotated track preview and named track sectors.

3.1 Track characteristics

The course occupies approximately $30\text{ m} \times 24\text{ m}$ and has a nominal width of about 2.3 m . It includes a long straight, fast sweepers, an S-shaped transition, a tighter eastern complex and a western recovery-sensitive section. Solid collision boundaries must be LiDAR-visible; decorative painted lines alone are not sufficient.

4 Computer-Vision Component

The vehicle receives images from a simulated forward-facing RGB camera. Track-side boards carry ArUco markers from the OpenCV `DICT_4X4_50` dictionary.

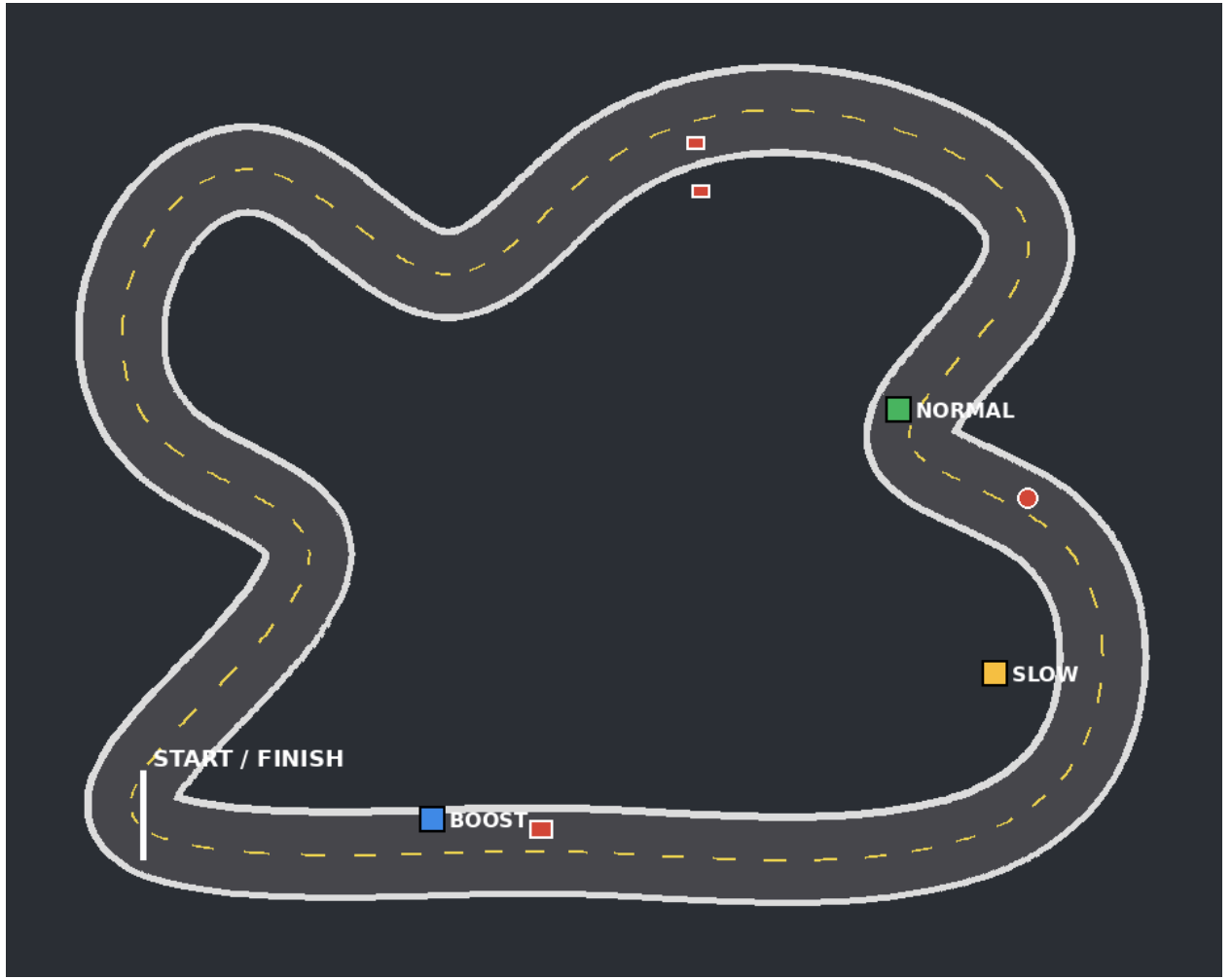


Figure 1: Development track. Red shapes are the public development obstacles. Colored squares indicate public visual-sign locations.

Marker ID	Sign	Nominal speed cap	Required response
10	SLOW	1.0 m/s	Reduce speed before the following complex
20	NORMAL	1.8 m/s	Restore the normal race-speed ceiling
30	BOOST	2.5 m/s	Permit higher speed where curvature and safety allow

The sign value is an *upper bound*, not a direct throttle command. The final speed must also satisfy curvature, acceleration and LiDAR constraints:

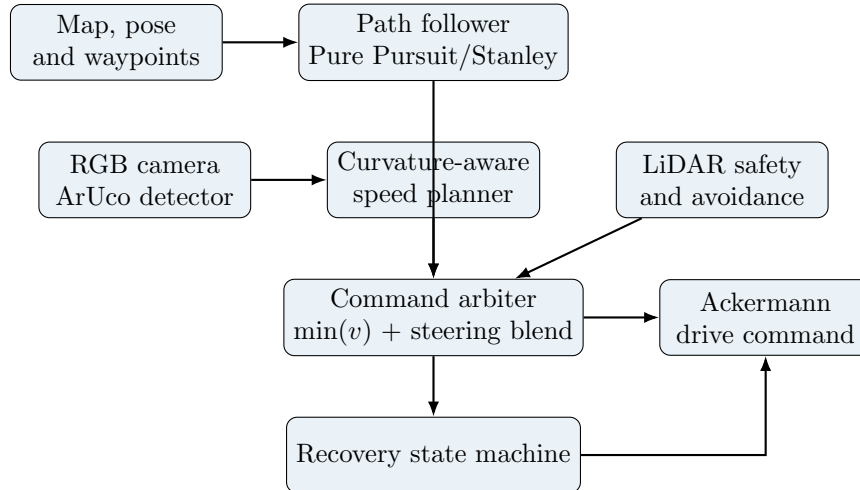
$$v_{\text{cmd}} = \min(v_{\text{curve}}, v_{\text{sign}}, v_{\text{lidar}}).$$

Vision requirements

- Detect marker ID and publish the active speed-zone state.
- Reject isolated false detections using temporal confirmation, confidence or repeated frames.
- Retain the most recently confirmed sign until a new valid sign is observed.
- Log timestamp, marker ID, detection confidence or reprojection measure, and active speed cap.
- Demonstrate operation under at least two image perturbations: brightness change, blur, partial occlusion or image noise.

Deep learning is not required. A correct OpenCV ArUco pipeline is sufficient and keeps the five-day scope realistic.

5 Recommended System Architecture



A suggested curvature speed law is

$$v_{\text{curve}} = \text{clip} \left(\frac{v_{\text{max}}}{1 + k_{\kappa} |\kappa|}, v_{\text{min}}, v_{\text{max}} \right).$$

Use acceleration and deceleration limits to avoid discontinuous commands.

6 Obstacle Challenge

Development obstacles are visible in the supplied layout. Evaluation uses a different instructor-selected layout. All layouts leave a feasible passage; no obstacle fully blocks the course.

The controller must:

- reduce speed as forward clearance decreases;
- trigger an emergency stop below a configured threshold;
- steer around partial blockages without crossing the track boundary;
- avoid oscillation in narrow passages;
- resume nominal racing after the obstacle is cleared.

The final evaluation can move obstacles and signs independently. Hard-coded actions tied to waypoint numbers will therefore receive poor robustness marks.

7 Five-Day Schedule and Checkpoints

Day 1: Platform, topics and baseline lap

Inspect the simulator interfaces, teleoperate the vehicle, visualize the map and run a supplied low-speed baseline. Implement waypoint loading and lap counting.

Deliverable: One collision-free low-speed lap, topic/interface diagram and a short baseline log.

Day 2: Path following

Implement Pure Pursuit, Stanley or another approved geometric controller. Tune look-ahead and steering limits. Complete three fixed-speed laps.

Deliverable: Controller code, parameter file and plot of lateral/heading error over one lap.

Day 3: Speed planning and vision

Estimate upcoming curvature and generate a smooth speed profile. Implement ArUco sign detection and integrate the active sign speed cap.

Deliverable: Three-sign detection demo, sign log, speed-profile plot and a lap showing at least two speed-zone transitions.

Day 4: LiDAR safety and recovery

Add obstacle-aware speed limiting, emergency braking, local avoidance, stuck detection and a recovery state machine. Test image perturbations and at least two forced recovery cases.

Deliverable: Obstacle run, recovery-state log, safety-threshold justification and vision robustness table.

Day 5: Final evaluation and presentation

Tune only on the public development configuration. Submit the final package before the hidden evaluation configuration is revealed. Each team receives three official runs.

Deliverable: Final repository, report, logs, plots, demonstration video and five-minute presentation.

8 Minimum Software Interface

The instructor should expose an equivalent interface even if exact topic names differ across simulator branches.

Inputs	
/scan	sensor_msgs/LaserScan
/camera/front/image_raw	sensor_msgs/Image
/odom or /pose	nav_msgs/Odometry or pose equivalent
Output	
/drive	ackermann_msgs/AckermannDriveStamped
Recommended team outputs	
/race/active_sign	std_msgs/Int32
/race/target_speed	std_msgs/Float32
/race/state	std_msgs/String
/race/lap_count	std_msgs/Int32

9 Rules and Constraints

- No teleoperation after the official run begins.
- The supplied centre-line may be smoothed or locally offset, but the map may not be modified.
- Hidden obstacle and sign positions must not be read directly from instructor configuration files at runtime.
- LiDAR must be used for collision safety; vision alone is insufficient for obstacle avoidance.
- Camera images must be processed online during the run.
- Pretrained neural networks are optional but provide no automatic bonus.
- A manual reset ends the current official run and incurs the reset penalty.

10 Evaluation Protocol

Round 1: Clean-track time trial

Three laps, no obstacles, public sign layout. Measures path tracking, speed planning and vision integration.

Round 2: Obstacle race

Three laps with development-style static obstacles and a changed sign layout. Measures local safety and non-memorized perception.

Round 3: Robustness run

One or two laps using an instructor-selected evaluation obstacle set, changed sign positions and mild camera perturbation.

Penalties

Event	Penalty
Obstacle contact	+10 s
Track-boundary contact	+5 s
Missed required sign response	+8 s each
False sign transition	+4 s each
Manual reset	+20 s and recovery score loss
Incomplete lap	Lap not scored

11 Grading Rubric

Component	Weight
Reliable autonomous completion and lap counting	20%
Path following and curvature-aware speed control	20%
LiDAR obstacle safety and local avoidance	18%
Computer-vision sign detection and robustness	17%
Recovery behaviour and fault handling	10%
Experimental analysis, plots and reproducibility	10%
Code quality and presentation	5%

To pass, a team must complete at least one autonomous lap, detect at least two marker classes online, and demonstrate an emergency stop. Racing speed cannot compensate for failure of these minimum safety and perception requirements.

12 Required Submission

Submit one compressed archive or repository containing:

1. ROS package(s), launch files and configuration files;
2. a README with installation and exact run commands;
3. a one-page architecture diagram;
4. controller, vision and recovery parameter files;
5. CSV logs for time, pose, speed, steering, minimum LiDAR range, active sign and state;
6. plots of lap speed, steering, tracking error and obstacle clearance;
7. a vision evaluation table with precision, recall or detection success rate;
8. a two-to-three-minute demonstration video;
9. a report of at most five pages excluding appendices;
10. a brief statement of each team member's contribution.

13 Suggested Report Questions

1. How was look-ahead distance selected, and should it vary with speed?
2. How was curvature estimated from discrete waypoints?
3. Which constraint most often limited speed: curvature, sign or LiDAR?
4. How did the marker detector reject false positives and duplicate observations?
5. What failure modes remained on the hidden configuration?
6. Which single change would most improve lap time without reducing safety?

14 Instructor Notes

The included map is a standard ROS-style occupancy grid rather than a simulator-specific world file. The exact MuSHR simulator branch may require a thin adapter for map loading, obstacle spawning, RGB camera publication and topic names. Provide students a tested launch command and baseline controller before Day 1. Keep evaluation layouts private and verify every generated obstacle set leaves a physically feasible route.

The vision task is deliberately fiducial-based. It teaches image transport, camera geometry, online detection, filtering and decision integration without requiring dataset collection or GPU training. For a more advanced cohort, replace one marker class with a conventional colored or symbolic sign and require a classical contour/classification pipeline.

15 References

1. S. S. Srinivasa et al., “MuSHR: A Low-Cost, Open-Source Robotic Racecar for Education and Research,” 2019.
2. MuSHR project repository, Personal Robotics Lab, University of Washington: <https://github.com/prl-mushr/mushr>.
3. OpenCV documentation, ArUco marker detection module: https://docs.opencv.org/4.x/d5/dae/tutorial_aruco_detection.html.